Python Handbook

From Zero to ML Ready

23 Days | Concepts + Code + Practice

May 9 - May 31, 2025

Day 1 + 2 | May 9 | Variables, Data Types, Type Casting, Input, Output, Operators

1. Variables

A variable is a name that stores a value in memory. Think of it as a labeled box.

```
# Creating variables
name = 'Rahul'
age = 20
height = 5.9
is_student = True

print(name)     # Rahul
print(age)      # 20
```

> *Note: Variable names are case-sensitive. 'Age' and 'age' are two different variables.*

2. Data Types

Python has several built-in data types. You do not declare the type, Python figures it out.

```
x = 10          # int
y = 3.14        # float
name = 'Ali'    # str
flag = True     # bool
z = 2 + 3j      # complex

print(type(x))  # <class 'int'>
print(type(y))  # <class 'float'>
print(type(name)) # <class 'str'>
```

isinstance() - Check the type safely

```
x = 42
print(isinstance(x, int))     # True
print(isinstance(x, str))     # False
print(isinstance(x, (int, float)))  # True - checks multiple types
```

3. Type Casting

Converting one data type to another. This is something you will use constantly.

```
# str to int
age = int('25')       # 25

# int to str
num = str(100)        # '100'

# str to float
price = float('9.99')  # 9.99

# int to float
x = float(5)          # 5.0

# float to int (cuts decimal, does NOT round)
y = int(9.99)         # 9
```

Common Mistakes

- x int('3.14') will throw an error. You must do int(float('3.14')) to convert '3.14' to int.
- x int(True) gives 1, int(False) gives 0. Booleans are secretly integers in Python.
- x Casting does not change the original variable. x = 5.9, int(x) gives 5 but x is still 5.9.

4. Input and Output

```
# Input always returns a string
name = input('Enter your name: ')
age = int(input('Enter your age: '))  # convert immediately

# print with sep and end
print('Hello', 'World', sep='-')     # Hello-World
print('Loading', end='...')          # Loading... (no newline)

# f-strings (use these always)
print(f'My name is {name} and I am {age} years old')

# Expressions inside f-strings
print(f'Next year I will be {age + 1}')
```

5. Operators

Arithmetic

```
a, b = 10, 3
print(a + b)   # 13  - addition
print(a - b)   # 7   - subtraction
print(a * b)   # 30  - multiplication
print(a / b)   # 3.333 - division (always float)
print(a // b)  # 3   - floor division
print(a % b)   # 1   - modulus (remainder)
```

```
print(a ** b)  # 1000 - exponent
```

Comparison

```
print(10 == 10)  # True
print(10 != 5)   # True
print(10 > 5)    # True
print(10 < 5)    # False
print(10 >= 10)  # True
print(10 <= 9)   # False
```

Logical

```
print(True and False)  # False
print(True or False)   # True
print(not True)        # False

age = 20
print(age > 18 and age < 30)  # True
```

Assignment Operators

```
x = 10
x += 5   # x = x + 5  -> 15
x -= 3   # x = x - 3  -> 12
x *= 2   # x = x * 2  -> 24
x //= 5  # x = x // 5 -> 4
```

Practice Problems

1. Take user's name and age as input. Print: 'Hello [name], you were born in [year].' Calculate birth year.
2. Build a simple calculator. Take two numbers and an operator (+, -, *, /) from user. Print the result.
3. Check if a number entered by user is even or odd. Use modulus operator.
4. Print the multiplication table of any number entered by user (1 to 10).
5. Take a temperature in Celsius, convert to Fahrenheit. Formula: F = (C x 9/5) + 32.

Day 3 | May 10 | Strings

What is a String?

A string is a sequence of characters. Once created, it cannot be changed (immutable). Every character has an index starting from 0.

```
name = 'Python'
#       P y t h o n
#       0 1 2 3 4 5   (positive index)
#      -6-5-4-3-2-1   (negative index)

print(name[0])    # P
print(name[-1])   # n
print(name[1:4])  # yth  (start:stop, stop not included)
print(name[::-1]) # nohtyP  (reverse)
```

String Methods

```
text = '  Hello World  '

print(text.strip())        # 'Hello World'  - removes spaces
print(text.lower())        # '  hello world  '
print(text.upper())        # '  HELLO WORLD  '
print(text.replace('World', 'Python'))  # '  Hello Python  '
print(text.find('World'))  # 8 (index where it starts, -1 if not found)
print(text.count('l'))     # 3

sentence = 'one,two,three'
print(sentence.split(','))  # ['one', 'two', 'three']

words = ['Hello', 'World']
print(' '.join(words))      # 'Hello World'

print('python'.startswith('py'))  # True
print('python'.endswith('on'))    # True
print('  '.isspace())             # True
print('abc123'.isalnum())        # True
```

f-Strings (use these, always)

```
name = 'Ali'
score = 95.678

print(f'Name: {name}')
print(f'Score: {score:.2f}')       # 95.68 - 2 decimal places
```

```
print(f'Score: {score:08.2f}')      # 00095.68 - padded
print(f'Double: {score * 2}')       # expressions work
print(f'Upper: {name.upper()}')     # methods work too
```

Common Mistakes

- x Strings are immutable. name[0] = 'J' will throw a TypeError. You must create a new string.
- x name.upper() does not change name. It returns a new string. You must store it: name = name.upper().
- x 'Hello' + 5 will throw an error. You must convert: 'Hello' + str(5).

Practice Problems

6. Take a string input. Reverse it without using slicing. Use a loop.
7. Count how many vowels (a,e,i,o,u) are in a given string. Case insensitive.
8. Check if a string is a palindrome (reads same forwards and backwards). Example: 'racecar'.
9. Capitalize the first letter of every word in a sentence without using .title() method.
10. Count how many times each character appears in a string. Print as: 'a: 3, b: 1 ...'

Day 4 | May 11 | Lists

What is a List?

A list is an ordered, mutable collection. You can store anything in a list: numbers, strings, even other lists. Lists use square brackets.

```
fruits = ['apple', 'banana', 'cherry']
mixed = [1, 'hello', 3.14, True]
nested = [[1, 2], [3, 4], [5, 6]]

print(fruits[0])        # apple
print(fruits[-1])       # cherry
print(fruits[0:2])      # ['apple', 'banana']
print(len(fruits))      # 3
```

List Methods

```
nums = [3, 1, 4, 1, 5, 9]

nums.append(2)          # adds 2 at end
nums.insert(0, 0)       # inserts 0 at index 0
nums.remove(1)          # removes first occurrence of 1
nums.pop()              # removes and returns last item
nums.pop(0)             # removes and returns item at index 0
nums.sort()             # sorts in place
nums.reverse()          # reverses in place
nums.count(1)           # counts occurrences of 1
nums.index(5)           # returns index of 5
nums.clear()            # removes all elements

copy1 = nums.copy()     # shallow copy
copy2 = nums[:]         # also shallow copy
```

Shallow vs Deep Copy

This matters when you have nested lists. A shallow copy copies the outer list but the inner lists are still shared.

```
import copy

original = [[1, 2], [3, 4]]
shallow = original.copy()
deep = copy.deepcopy(original)
```

```
original[0][0] = 99

print(shallow)    # [[99, 2], [3, 4]] - affected!
print(deep)       # [[1, 2], [3, 4]]  - not affected
```

Common Mistakes

- x list2 = list1 does NOT copy a list. Both names point to the same list. Use list1.copy() or list1[:].
- x list.sort() modifies the list in place and returns None. sorted(list) returns a new sorted list.
- x Removing items from a list while iterating over it will cause bugs. Iterate over a copy instead.

Practice Problems

11. Remove all duplicate values from a list without using set(). Preserve the original order.
12. Find the second largest number in a list without using sort() or sorted().
13. Flatten this list without using any library: [[1,2],[3,[4,5]],[6]]. Output: [1,2,3,4,5,6].
14. Rotate a list to the right by k positions. Example: [1,2,3,4,5], k=2 -> [4,5,1,2,3].
15. Merge two sorted lists into one sorted list without using sort().

Day 5 | May 13 | Tuples & Sets

Tuples

A tuple is like a list but immutable. Once created, you cannot change it. Use tuples for data that should not change: coordinates, RGB values, database records.

```
point = (3, 7)
colors = ('red', 'green', 'blue')
single = (42,)   # single element tuple needs trailing comma

# Unpacking
x, y = point
print(x, y)     # 3 7

# Swap using tuple unpacking
a, b = 10, 20
a, b = b, a
print(a, b)     # 20 10

# Tuple methods
print(colors.index('green'))  # 1
print(colors.count('red'))    # 1
```

Sets

A set is an unordered collection with no duplicates. Useful for membership testing and removing duplicates.

```
fruits = {'apple', 'banana', 'cherry', 'apple'}
print(fruits)   # {'apple', 'banana', 'cherry'} - duplicate removed

# Set operations
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

print(a | b)    # {1,2,3,4,5,6}  - union
print(a & b)    # {3,4}          - intersection
print(a - b)    # {1,2}          - difference
print(a ^ b)    # {1,2,5,6}      - symmetric difference

# Membership (much faster than list for large data)
print('apple' in fruits)  # True

fruits.add('mango')
fruits.remove('banana')   # KeyError if not found
fruits.discard('kiwi')    # no error if not found
```

Note: Sets are unordered. You cannot access elements by index. If you need order, use a list.

Common Mistakes

- x Empty set must be set(), not {}. Empty {} creates a dictionary, not a set.
- x Sets cannot contain mutable items. You cannot put a list inside a set.
- x Tuples with one element need a trailing comma: (42,) not (42). Without it, it is just an integer in parentheses.

Practice Problems

16. Swap two variables using tuple unpacking. Do it in one line.
17. Find common elements between two lists using sets. Then find elements only in list1 but not list2.
18. Remove duplicates from a list using a set, but preserve the original order.
19. Given a list of numbers, use set operations to find numbers that appear in exactly one of two lists.

Day 6 | May 14 | Dictionaries

What is a Dictionary?

A dictionary stores data as key-value pairs. Keys must be unique and immutable. Values can be anything. This is one of the most used data structures in Python.

```
student = {
    'name': 'Arjun',
    'age': 21,
    'marks': [85, 90, 78]
}

print(student['name'])           # Arjun
print(student.get('age'))        # 21
print(student.get('gpa', 0.0)) # 0.0 (default if key missing)

# Add / Update
student['email'] = 'arjun@mail.com'
student['age'] = 22

# Delete
del student['email']
student.pop('age')  # removes and returns value
```

Iterating Dictionaries

```
info = {'a': 1, 'b': 2, 'c': 3}

for key in info:
    print(key)

for value in info.values():
    print(value)

for key, value in info.items():
    print(f'{key} -> {value}')
```

Nested Dictionaries

```
students = {
    'Rahul': {'age': 20, 'marks': 88},
    'Priya': {'age': 21, 'marks': 95},
}

print(students['Priya']['marks'])  # 95
```

```
# Adding new student
students['Amit'] = {'age': 19, 'marks': 76}
```

Common Mistakes

- x student['gpa'] will raise KeyError if 'gpa' does not exist. Always use .get() when unsure.
- x Dictionary keys must be immutable. You can use str, int, tuple as keys. Not list.
- x Iterating and modifying a dict at the same time raises RuntimeError. Iterate over list(dict.keys()) instead.

Practice Problems

20. Build a contact book. User can: add a contact (name+phone), search by name, delete a contact, show all contacts.
21. Count the frequency of each word in a sentence. Example: 'hello world hello' -> {'hello':2, 'world':1}.
22. Merge two dictionaries. If same key exists, the value from second dict should win.
23. Given a dict of student names and marks, find the topper and the average marks.

Day 7 | May 15 | Revision + Mini Project

Revision Checklist

Before starting the project, go through each point. If any feels unclear, revisit that day.

```
# Quick revision test - write output without running

x = '5'
print(type(int(x)))          # ?

a = [1, 2, 3]
b = a
b.append(4)
print(a)                     # ?

s = {1, 2, 3} & {2, 3, 4}
print(s)                     # ?

d = {'a': 1}
print(d.get('b', 99))        # ?
```

Mini Project: Student Grade Manager

Build this step by step. Do not look for shortcuts. Use only what you have learned so far.

```
# Requirements:
# 1. Store multiple students with their name and list of marks
# 2. Calculate average marks for each student
# 3. Assign grade: A(90+), B(75+), C(60+), D(below 60)
# 4. Find the topper (highest average)
# 5. Print a formatted report for all students

# Example output:
# Name     | Average | Grade
# Rahul    | 88.3    | B
# Priya    | 94.0    | A   <- Topper
# Amit     | 57.5    | D
```

> *Note: If you finish in under 30 minutes, you are moving too fast and likely missing something. This project should take 45-60 minutes if you are doing it properly.*

Day 8 | May 16 | Conditional Statements

if, elif, else

```
age = int(input('Enter age: '))

if age >= 18:
    print('Adult')
elif age >= 13:
    print('Teenager')
else:
    print('Child')
```

Nested Conditions

```
score = 85
attendance = 90

if score >= 60:
    if attendance >= 75:
        print('Pass')
    else:
        print('Fail - low attendance')
else:
    print('Fail - low score')
```

Ternary Operator (one-liner if-else)

```
age = 20
status = 'Adult' if age >= 18 else 'Minor'
print(status)  # Adult

# Nested ternary (avoid if possible, hard to read)
grade = 'A' if score >= 90 else 'B' if score >= 75 else 'C'
```

Common Mistakes

- x Python uses indentation to define blocks. Missing indentation will throw IndentationError.
- x = is assignment, == is comparison. if x = 5 is wrong. Use if x == 5.
- x if x == True can just be written as if x. Same for if x == False -> if not x.

Practice Problems

24. Build a grade calculator. Take marks as input, print: A (90+), B (75-89), C (60-74), D (below 60).
25. Check if a year is a leap year. Rules: divisible by 4, but not 100, unless also divisible by 400.
26. Find the biggest of 3 numbers without using max() function.
27. Take a number. Print 'Positive', 'Negative', or 'Zero'. Also print if it is even or odd (for non-zero).

Day 9 | May 17 | Loops

for Loop

Use for loop when you know how many times to iterate or when iterating over a sequence.

```
# Iterating a list
fruits = ['apple', 'banana', 'cherry']
for fruit in fruits:
    print(fruit)

# range(start, stop, step)
for i in range(5):        # 0,1,2,3,4
    print(i)

for i in range(1, 10, 2): # 1,3,5,7,9
    print(i)

# enumerate - get index and value
for i, fruit in enumerate(fruits):
    print(f'{i}: {fruit}')

# zip - iterate two lists together
names = ['Ali', 'Sara']
scores = [88, 95]
for name, score in zip(names, scores):
    print(f'{name}: {score}')
```

while Loop

Use while loop when you do not know how many iterations you need. Runs as long as condition is True.

```
count = 0
while count < 5:
    print(count)
    count += 1

# Guess the number game
secret = 7
guess = int(input('Guess: '))
while guess != secret:
    if guess < secret:
        print('Too low')
    else:
        print('Too high')
    guess = int(input('Guess again: '))
print('Correct!')
```

Common Mistakes

- x Forgetting to update the loop variable in a while loop creates infinite loop. Always update it.
- x range(5) gives 0 to 4, not 0 to 5. The stop value is excluded.
- x for i in range(len(list)) works but for item in list is cleaner and more Pythonic.

Practice Problems

28. Print all prime numbers between 1 and 100.
29. Sum of digits of a number. Example: 1234 -> 1+2+3+4 = 10.
30. Reverse a number using while loop. Example: 1234 -> 4321.
31. Find all factors of a number entered by user.

Day 10 | May 18 | Nested Loops & Loop Control

Nested Loops

```
# Multiplication table
for i in range(1, 4):
    for j in range(1, 4):
        print(f'{i*j:3}', end=' ')
    print()

# Triangle pattern
n = 5
for i in range(1, n+1):
    for j in range(i):
        print('*', end=' ')
    print()
```

break, continue, pass

```
# break - exits the loop entirely
for i in range(10):
    if i == 5:
        break
    print(i)  # prints 0,1,2,3,4

# continue - skips current iteration
for i in range(10):
    if i % 2 == 0:
        continue
    print(i)  # prints odd numbers only

# pass - does nothing, placeholder
for i in range(5):
    if i == 3:
        pass  # will add logic later
    print(i)

# Loop else - runs if loop completed without break
for i in range(5):
    print(i)
else:
    print('Loop finished normally')
```

Practice Problems

32. Print a diamond pattern of stars. If n=3: print 1 star, 3 stars, 5 stars, then back down.
33. Find all pairs in a list that sum to a target number. Example: [1,2,3,4], target=5 -> (1,4),(2,3).

34. FizzBuzz: Print 1 to 100. For multiples of 3 print Fizz, for 5 print Buzz, for both print FizzBuzz.
35. Print a number triangle where each row contains row number repeated. Row 3: 3 3 3.

Day 11 | May 19 | Functions

Defining Functions

A function is a reusable block of code. Write once, use many times. Functions make code clean, readable, and testable.

```
def greet(name):
    return f'Hello, {name}!'

print(greet('Rahul'))  # Hello, Rahul!

# Multiple return values
def min_max(nums):
    return min(nums), max(nums)

low, high = min_max([3, 1, 9, 5])
print(low, high)  # 1 9
```

Default Parameters & Keyword Arguments

```
def power(base, exp=2):   # exp defaults to 2
    return base ** exp

print(power(3))        # 9  (3^2)
print(power(3, 3))     # 27 (3^3)
print(power(exp=3, base=2))  # keyword args, order does not matter
```

Scope

```
x = 10  # global

def func():
    x = 20  # local - different variable
    print(x)  # 20

func()
print(x)  # 10 - global unchanged

# To modify global variable
def func2():
    global x
    x = 99

func2()
print(x)  # 99
```

Common Mistakes

- x Default mutable arguments (like lists) are shared across calls. Use None as default, create inside.
- x A function without return statement returns None. Do not assign it expecting a value.
- x Variables defined inside a function do not exist outside. Accessing them raises NameError.

Practice Problems

36. Write a function is_prime(n) that returns True if n is prime, False otherwise.
37. Write a function fibonacci(n) that returns the nth Fibonacci number using recursion.
38. Build a simple ATM system with functions: deposit(amount), withdraw(amount), check_balance(). Use a global balance variable.
39. Write a function that takes a list and returns a new list with duplicates removed, preserving order.

Day 12 | May 20 | Advanced Functions

*args and **kwargs

```
# *args - accepts any number of positional arguments
def total(*args):
    return sum(args)

print(total(1, 2, 3))        # 6
print(total(1, 2, 3, 4, 5)) # 15

# **kwargs - accepts any number of keyword arguments
def user_info(**kwargs):
    for key, value in kwargs.items():
        print(f'{key}: {value}')

user_info(name='Ali', age=22, city='Delhi')

# Combining all
def func(a, b, *args, **kwargs):
    print(a, b, args, kwargs)

func(1, 2, 3, 4, x=10, y=20)
# Output: 1 2 (3, 4) {'x': 10, 'y': 20}
```

Lambda Functions

A lambda is a small anonymous function. It can only have one expression. Used for short operations passed as arguments.

```
# Regular function
def square(x):
    return x ** 2

# Same as lambda
square = lambda x: x ** 2
print(square(5))  # 25

# Sorting with lambda
students = [{'name': 'Ali', 'marks': 88}, {'name': 'Sara', 'marks': 95}]
students.sort(key=lambda s: s['marks'], reverse=True)
print(students)  # Sara first
```

Map, Filter, Zip

```
nums = [1, 2, 3, 4, 5]
```

```
# map - apply function to each element
squares = list(map(lambda x: x**2, nums))
print(squares)  # [1, 4, 9, 16, 25]

# filter - keep elements where function returns True
evens = list(filter(lambda x: x % 2 == 0, nums))
print(evens)  # [2, 4]

# zip - combine two lists
names = ['Ali', 'Sara', 'Raj']
scores = [88, 95, 72]
combined = list(zip(names, scores))
print(combined)  # [('Ali',88), ('Sara',95), ('Raj',72)]
```

Practice Problems

40. Write a function that accepts any number of numbers and returns their average using *args.
41. Sort a list of tuples (name, age) by age using lambda.
42. Use map() to convert a list of temperatures in Celsius to Fahrenheit.
43. Use filter() to get all words longer than 4 characters from a list of words.
44. Implement binary search using recursion.

Day 13 | May 21 | Comprehensions

List Comprehension

A compact way to create lists. Replaces a for loop + append in one line.

```
# Old way
squares = []
for x in range(10):
    squares.append(x**2)

# List comprehension
squares = [x**2 for x in range(10)]

# With condition
even_squares = [x**2 for x in range(10) if x % 2 == 0]
print(even_squares)  # [0, 4, 16, 36, 64]

# Nested
matrix = [[i*j for j in range(1,4)] for i in range(1,4)]
print(matrix)  # [[1,2,3],[2,4,6],[3,6,9]]
```

Dict & Set Comprehension

```
# Dict comprehension
words = ['hello', 'world', 'python']
word_lengths = {word: len(word) for word in words}
print(word_lengths)  # {'hello':5, 'world':5, 'python':6}

# Invert a dictionary
original = {'a': 1, 'b': 2, 'c': 3}
inverted = {v: k for k, v in original.items()}
print(inverted)  # {1:'a', 2:'b', 3:'c'}

# Set comprehension
nums = [1, 2, 2, 3, 3, 3, 4]
unique_squares = {x**2 for x in nums}
print(unique_squares)  # {1, 4, 9, 16}
```

Generator Expression

```
# List comprehension - creates full list in memory
squares_list = [x**2 for x in range(1000000)]  # uses ~8MB

# Generator expression - lazy, one at a time
squares_gen = (x**2 for x in range(1000000))   # uses ~100 bytes
```

```
# You can iterate over it
for val in squares_gen:
    print(val)
    break  # prints just 0

# sum works directly with generator
total = sum(x**2 for x in range(100))
```

Practice Problems

45. Use list comprehension to get squares of even numbers from 1 to 50.
46. Use dict comprehension to create a dict of number: cube for numbers 1 to 10.
47. Filter words longer than 4 letters from a sentence using list comprehension.
48. Flatten a 2D list using list comprehension: [[1,2],[3,4],[5,6]] -> [1,2,3,4,5,6].
49. Compare memory: print size of a list comprehension vs generator expression for range(100000). Use sys.getsizeof().

Day 14 | May 22 | Revision + Mini Project

Mini Project: Word Analysis Tool

This project uses functions, loops, dicts, comprehensions, and strings together. Build it properly.

```
# Requirements:
# 1. Take a paragraph as input from user
# 2. Count total words
# 3. Count unique words (case insensitive)
# 4. Find the most frequent word
# 5. Remove common stopwords: ['the','a','an','is','in','of','and','to']
# 6. Find top 5 words by frequency after removing stopwords
# 7. Print word lengths using dict comprehension

# Bonus:
# 8. Count sentences (split by . ! ?)
# 9. Average word length
# 10. Find all words that appear exactly once
```

> *Note: This should take you 60-90 minutes. If it takes less, you are probably skipping requirements. If it takes more than 2 hours, go back and review functions and dicts.*

Day 15 | May 23 | Modules, Packages & Virtual Environments

Built-in Modules

```
import math
print(math.pi)          # 3.14159...
print(math.sqrt(16))    # 4.0
print(math.ceil(4.2))   # 5
print(math.floor(4.9))  # 4

import random
print(random.randint(1, 10))        # random int between 1-10
print(random.choice(['a','b','c'])) # random choice
items = [1,2,3,4,5]
random.shuffle(items)               # shuffle in place

import datetime
today = datetime.date.today()
print(today)                        # 2025-05-23
print(today.year, today.month)      # 2025 5

import os
print(os.getcwd())         # current directory
print(os.listdir('.'))     # list files in current dir
```

Creating Your Own Module

```
# File: mymath.py
def add(a, b):
    return a + b

def is_prime(n):
    if n < 2: return False
    for i in range(2, int(n**0.5)+1):
        if n % i == 0: return False
    return True

PI = 3.14159

# In another file: main.py
import mymath
print(mymath.add(3, 4))     # 7
print(mymath.is_prime(7))   # True
print(mymath.PI)            # 3.14159

# Or import specific things
from mymath import add, PI
print(add(1, 2))  # 3
```

Virtual Environments & Pip

```
# Create a virtual environment
python -m venv myenv

# Activate (Windows)
myenv\Scripts\activate

# Activate (Mac/Linux)
source myenv/bin/activate

# Install packages
pip install numpy pandas

# See installed packages
pip list

# Save requirements
pip freeze > requirements.txt

# Install from requirements
pip install -r requirements.txt

# Deactivate
deactivate
```

> *Note: Always use a virtual environment for each project. Never install packages globally. This prevents version conflicts between projects.*

Practice Problems

50. Generate a 6-digit OTP using the random module. Make it always 6 digits (pad with zeros if needed).
51. Get today's date and calculate how many days until December 31st of this year.
52. Create a module called 'converter.py' with functions: celsius_to_fahrenheit, km_to_miles, kg_to_lbs. Import and use it.
53. Use os module to list all .py files in a folder.

Day 16 | May 24 | File Handling & Exception Handling

File Handling

```
# Writing to a file
with open('notes.txt', 'w') as f:
    f.write('Hello World\n')
    f.write('Second line\n')

# Reading from a file
with open('notes.txt', 'r') as f:
    content = f.read()       # entire file as string
    # OR
    lines = f.readlines()    # list of lines
    # OR
    for line in f:           # line by line (memory efficient)
        print(line.strip())

# Append to existing file
with open('notes.txt', 'a') as f:
    f.write('New line added\n')
```

> *Note: Always use 'with' statement for file handling. It automatically closes the file even if an error occurs.*

Exception Handling

```
# Basic try-except
try:
    num = int(input('Enter number: '))
    result = 10 / num
    print(result)
except ValueError:
    print('That is not a number')
except ZeroDivisionError:
    print('Cannot divide by zero')
except Exception as e:
    print(f'Something went wrong: {e}')
finally:
    print('This always runs')

# Custom exceptions
class AgeError(Exception):
    pass

def set_age(age):
    if age < 0 or age > 150:
        raise AgeError(f'Invalid age: {age}')
    return age

try:
```

```
    set_age(-5)
except AgeError as e:
    print(e)  # Invalid age: -5
```

Practice Problems

54. Read a text file and count: total lines, total words, total characters.
55. Build a file-based note app. User can: add a note, view all notes, delete a note by number. Store in notes.txt.
56. Write a program that logs any ZeroDivisionError or ValueError to an error.log file with timestamp.
57. Read a CSV file manually (no pandas). Split by comma, print each row as a formatted table.

Day 17 | May 25 | OOP Part 1 - Classes, Objects, Methods

Why OOP?

OOP lets you model real-world things as code objects. A Car has attributes (color, speed) and behaviors (drive, stop). Instead of managing separate variables and functions, you group everything into one class.

Classes and Objects

```
class Car:
    # Class variable - shared by all cars
    total_cars = 0

    def __init__(self, brand, color, speed):
        # Instance variables - unique to each car
        self.brand = brand
        self.color = color
        self.speed = speed
        Car.total_cars += 1

    def drive(self):
        print(f'{self.brand} is driving at {self.speed} km/h')

    def repaint(self, new_color):
        self.color = new_color
        print(f'{self.brand} repainted to {new_color}')

# Creating objects
car1 = Car('Toyota', 'Red', 120)
car2 = Car('BMW', 'Black', 200)

car1.drive()              # Toyota is driving at 120 km/h
car1.repaint('Blue')
print(Car.total_cars)     # 2
```

BankAccount Example

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance
        self.transactions = []

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError('Amount must be positive')
        self.balance += amount
```

```
        self.transactions.append(f'+{amount}')

    def withdraw(self, amount):
        if amount > self.balance:
            raise ValueError('Insufficient funds')
        self.balance -= amount
        self.transactions.append(f'-{amount}')

    def statement(self):
        print(f'Account: {self.owner}')
        print(f'Balance: {self.balance}')
        print('Transactions:', self.transactions)

acc = BankAccount('Rahul', 1000)
acc.deposit(500)
acc.withdraw(200)
acc.statement()
```

Practice Problems

58. Create a Student class with name, roll_no, marks list. Add methods: add_marks(), average(), grade().
59. Create a Library class that stores books (title, author). Add methods: add_book(), remove_book(), search_by_author(), show_all().
60. Track how many Student objects have been created using a class variable.
61. Create a Rectangle class with length and width. Add methods: area(), perimeter(), is_square().

Day 18 | May 26 | OOP Part 2 - Inheritance & Polymorphism

Inheritance

A child class inherits all attributes and methods from a parent class. You extend or override behavior without rewriting code.

```
class Animal:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def eat(self):
        print(f'{self.name} is eating')

    def speak(self):
        print('...')

class Dog(Animal):
    def __init__(self, name, age, breed):
        super().__init__(name, age)  # call parent __init__
        self.breed = breed

    def speak(self):  # override parent method
        print(f'{self.name} says: Woof!')

    def fetch(self):
        print(f'{self.name} fetches the ball')

class Cat(Animal):
    def speak(self):
        print(f'{self.name} says: Meow!')

dog = Dog('Rex', 3, 'Labrador')
cat = Cat('Whiskers', 2)

dog.eat()      # inherited from Animal
dog.speak()    # overridden in Dog
dog.fetch()    # Dog-specific
```

Polymorphism

Same method name, different behavior depending on the object. This is what makes code flexible.

```
animals = [Dog('Rex', 3, 'Lab'), Cat('Mimi', 2), Animal('Bird', 1)]

for animal in animals:
    animal.speak()  # each behaves differently
```

```
# Output:
# Rex says: Woof!
# Mimi says: Meow!
# ...

# isinstance check
print(isinstance(dog, Dog))     # True
print(isinstance(dog, Animal))  # True - dog is also an Animal
print(isinstance(cat, Dog))     # False
```

Practice Problems

62. Create Employee base class with name, salary, work() method. Create Manager and Developer subclasses. Override work() in each.
63. Create Shape base class. Rectangle and Circle inherit from it. Each implements area() and perimeter() differently.
64. Create a Vehicle class. Car and Bike inherit from it. Add a fuel_type attribute and override start() method.
65. Create a list of different Employee objects. Loop and call work() on each to demonstrate polymorphism.

Day 19 | May 27 | OOP Part 3 - Encapsulation, Dunder Methods

Encapsulation

Hiding internal data and only exposing what is necessary. Private variables use double underscore prefix.

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.__marks = marks  # private variable

    @property
    def marks(self):          # getter
        return self.__marks

    @marks.setter
    def marks(self, value):   # setter with validation
        if value < 0 or value > 100:
            raise ValueError('Marks must be 0-100')
        self.__marks = value

s = Student('Ali', 85)
print(s.marks)    # 85 - uses getter
s.marks = 90      # uses setter
s.marks = 150     # raises ValueError
```

Dunder (Magic) Methods

```
class Book:
    def __init__(self, title, pages):
        self.title = title
        self.pages = pages

    def __str__(self):
        return f'Book: {self.title}'

    def __repr__(self):
        return f"Book('{self.title}', {self.pages})"

    def __len__(self):
        return self.pages

    def __add__(self, other):
        return Book(self.title + ' + ' + other.title,
                    self.pages + other.pages)

    def __eq__(self, other):
        return self.pages == other.pages
```

```
b1 = Book('Python', 300)
b2 = Book('ML', 450)

print(b1)           # Book: Python
print(len(b1))      # 300
b3 = b1 + b2
print(b3.title)     # Python + ML
print(b1 == b2)     # False
```

Practice Problems

66. Build a full Library Management System: Book class, Library class. Library has add, remove, search, show_all. Books have title, author, year, available status.
67. Create a Temperature class with @property for celsius, fahrenheit, kelvin. Setting one should auto-update the others.
68. Create a Vector class with __add__, __sub__, __mul__ (scalar), __len__, __str__ methods.

Day 20 | May 28 | Iterators, Generators & Decorators

Iterators

```
# Building a custom iterator
class CountUp:
    def __init__(self, start, end):
        self.current = start
        self.end = end

    def __iter__(self):
        return self

    def __next__(self):
        if self.current > self.end:
            raise StopIteration
        val = self.current
        self.current += 1
        return val

for num in CountUp(1, 5):
    print(num)  # 1, 2, 3, 4, 5
```

Generators

```
# Generator function
def countdown(n):
    while n > 0:
        yield n
        n -= 1

for num in countdown(5):
    print(num)  # 5, 4, 3, 2, 1

# Generator for reading large file
def read_chunks(filepath, chunk_size=1024):
    with open(filepath, 'r') as f:
        while True:
            chunk = f.read(chunk_size)
            if not chunk:
                break
            yield chunk

# Memory efficient - reads one chunk at a time
for chunk in read_chunks('bigfile.txt'):
    process(chunk)
```

Decorators

```
import time

# Timer decorator
def timer(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f'{func.__name__} took {end-start:.4f}s')
        return result
    return wrapper

@timer
def slow_function():
    time.sleep(1)
    return 'done'

slow_function()  # slow_function took 1.0001s

# Logger decorator
def logger(func):
    def wrapper(*args, **kwargs):
        print(f'Calling {func.__name__} with {args}')
        result = func(*args, **kwargs)
        print(f'{func.__name__} returned {result}')
        return result
    return wrapper

@logger
def add(a, b):
    return a + b

add(3, 5)
```

Practice Problems

69. Build a custom range iterator that counts by a step value, similar to Python's range().
70. Write a generator that produces Fibonacci numbers indefinitely. Stop after taking first 20.
71. Write a timer decorator. Apply it to a function that sorts a large list. Print time taken.
72. Write a retry decorator that retries a function up to 3 times if it raises an exception.

Day 21 | May 29 | NumPy

Why NumPy?

Python lists are slow for math. NumPy arrays are 10-100x faster because they store data in contiguous memory and use C under the hood. Every ML library is built on NumPy.

```
import numpy as np

# Creating arrays
a = np.array([1, 2, 3, 4, 5])
b = np.zeros((3, 3))          # 3x3 array of zeros
c = np.ones((2, 4))           # 2x4 array of ones
d = np.arange(0, 10, 2)       # [0, 2, 4, 6, 8]
e = np.linspace(0, 1, 5)      # [0, 0.25, 0.5, 0.75, 1.0]
f = np.random.randint(0, 10, (3,3))  # random 3x3 int array

print(a.shape)    # (5,)
print(b.shape)    # (3, 3)
print(a.dtype)    # int64
print(a.ndim)     # 1
```

Indexing, Slicing, Reshaping

```
arr = np.array([[1,2,3],[4,5,6],[7,8,9]])

print(arr[0])       # [1, 2, 3] - first row
print(arr[1][2])    # 6
print(arr[1, 2])    # 6 - same, cleaner
print(arr[:, 1])    # [2, 5, 8] - second column
print(arr[0:2, 0:2])  # top-left 2x2 subarray

# Boolean indexing
print(arr[arr > 5])  # [6, 7, 8, 9]

# Reshaping
flat = np.arange(1, 10)
matrix = flat.reshape(3, 3)
print(matrix)
```

Operations & Broadcasting

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(a + b)    # [5, 7, 9]
```

```
print(a * b)     # [4, 10, 18]
print(a ** 2)    # [1, 4, 9]
print(a + 10)    # [11, 12, 13]  - broadcasting scalar

# Statistical operations
data = np.array([2, 4, 6, 8, 10])
print(np.mean(data))     # 6.0
print(np.std(data))      # 2.83
print(np.min(data))      # 2
print(np.max(data))      # 10
print(np.sum(data))      # 30

# Matrix multiplication
A = np.array([[1,2],[3,4]])
B = np.array([[5,6],[7,8]])
print(np.dot(A, B))    # matrix multiplication
print(A @ B)           # same, cleaner
```

Practice Problems

73. Create a 5x5 array with random integers 1-100. Find: min, max, mean, row-wise sum, column-wise sum.
74. Normalize an array so all values are between 0 and 1. Formula: (x - min) / (max - min).
75. Create two 3x3 matrices and compute their matrix product without using a loop.
76. Find all rows in a 2D array where the row sum exceeds 15.
77. Generate 1000 random numbers from normal distribution. Count how many fall between -1 and 1.

Day 22 | May 30 | Pandas

Why Pandas?

Pandas is for structured data. Think of it as Excel in Python but programmable, faster, and much more powerful. Everything in ML starts with Pandas.

```
import pandas as pd

# Creating a DataFrame
data = {
    'name': ['Ali', 'Sara', 'Raj', 'Priya'],
    'age': [22, 25, 21, 23],
    'score': [88, 95, 72, 91],
    'city': ['Delhi', 'Mumbai', 'Delhi', 'Pune']
}
df = pd.DataFrame(data)

# Reading a CSV
df = pd.read_csv('data.csv')

# Quick exploration
print(df.head())          # first 5 rows
print(df.tail(3))         # last 3 rows
print(df.shape)           # (rows, cols)
print(df.info())          # dtypes and null counts
print(df.describe())      # statistics
print(df.columns)         # column names
```

Selecting & Filtering

```
# Select column
print(df['name'])                    # Series
print(df[['name', 'score']])         # DataFrame

# Select rows
print(df.iloc[0])                    # first row by index
print(df.loc[0])                     # first row by label
print(df.iloc[1:3])                  # rows 1 and 2

# Filter rows
print(df[df['score'] > 85])
print(df[(df['score'] > 80) & (df['city'] == 'Delhi')])

# Sorting
print(df.sort_values('score', ascending=False))
```

Cleaning & Transforming

```
# Handle missing values
print(df.isnull().sum())              # count nulls per column
df.dropna(inplace=True)              # drop rows with any null
df['age'].fillna(df['age'].mean())   # fill with mean

# Add new column
df['grade'] = df['score'].apply(lambda x: 'A' if x >= 90 else 'B' if x >= 75 else
'C')

# GroupBy
print(df.groupby('city')['score'].mean())
print(df.groupby('city').agg({'score': ['mean', 'max'], 'age': 'mean'}))

# Value counts
print(df['city'].value_counts())

# Rename columns
df.rename(columns={'name': 'student_name'}, inplace=True)
```

Practice Problems

78. Download the Titanic dataset from Kaggle. Load it and answer: How many passengers survived? What was average age? Any nulls?
79. Filter passengers who survived and were in first class. What is their average fare?
80. Group by 'Pclass' and find survival rate for each class.
81. Add a new column 'age_group': Child (under 18), Adult (18-60), Senior (60+) using apply().
82. Find the top 5 most expensive fares. Who paid them?

Day 23 | May 31 | Matplotlib & Seaborn + What's Next

Matplotlib Basics

```
import matplotlib.pyplot as plt
import numpy as np

# Line plot
x = [1, 2, 3, 4, 5]
y = [2, 4, 1, 6, 3]
plt.plot(x, y, color='blue', marker='o', linestyle='--')
plt.title('My Line Plot')
plt.xlabel('X axis')
plt.ylabel('Y axis')
plt.show()

# Bar chart
names = ['Ali', 'Sara', 'Raj']
scores = [88, 95, 72]
plt.bar(names, scores, color=['red','green','blue'])
plt.title('Student Scores')
plt.show()

# Subplots
fig, axes = plt.subplots(2, 2, figsize=(10, 8))
axes[0,0].plot(x, y)
axes[0,1].bar(names, scores)
axes[1,0].scatter(x, y)
axes[1,1].hist(np.random.randn(100), bins=20)
plt.tight_layout()
plt.show()
```

Seaborn

```
import seaborn as sns
import pandas as pd

df = sns.load_dataset('titanic')  # built-in dataset

# Heatmap (correlation)
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.show()

# Countplot
sns.countplot(data=df, x='class', hue='survived')
plt.show()

# Boxplot
sns.boxplot(data=df, x='class', y='age')
plt.show()
```

```
# Pairplot (all feature relationships)
sns.pairplot(df[['age','fare','survived']], hue='survived')
plt.show()
```

Practice Problems

83. Plot marks of 10 students as a bar chart. Add labels, title, colors.
84. Plot a histogram of 500 random normal numbers. Add a vertical line at the mean.
85. Load Titanic dataset with Seaborn. Create a 2x2 subplot: age distribution, class counts, survival by class, fare boxplot.
86. Create a heatmap of the correlation matrix on the Titanic dataset. Identify the top 3 correlated pairs.

What's Next After May 31

You finished Python foundations. Here is what comes next in order:

```
Phase 2 (after this handbook):

1. Scikit-learn          - ML models, pipelines, evaluation
2. Statistics basics     - mean, std, distributions, hypothesis testing
3. Linear Regression     - your first ML model from scratch
4. Logistic Regression   - classification
5. Decision Trees        - and Random Forests
6. Model Evaluation      - accuracy, precision, recall, F1
7. Cross Validation      - proper model validation
8. Feature Engineering   - making data better
9. Real Project          - end to end ML project on Kaggle
```

> *Note: This handbook is your reference. Come back to it whenever you forget something. The goal was never to memorize. It was to build the thinking.*